RX / SPORTS ANALYTICS

WHP Implementation Guide

NBA, NFL and NCAAF player models
From source observations to market probabilities

LOOKBACK / NEW SEASONS / CODE LOGIC / PRICING

A guide to the implementation in this repository, including the exact history windows, minimum samples, seasonal carryover, chronological calibration, saved forecast behavior and limitations.

Read section 0 first for the answers to how many games are used and what happens when a new season has little or no history.

The original research PDF is retained separately as a source reference. This manual explains what the code actually implements; advanced proposals in the source papers are not represented as completed features.

Code review date: 20 October 2026
Documentation source: models/Player_Stats/Weighted_Historical_Performance/README.md

Contents

0. What runs, how much history, and new seasons 3

1. Package boundaries and execution flow 4

2. Source observations and preparation 4

3. Cutoffs, identity matching and data quality 5

4. Recency weighting and weighted dispersion 6

5. Workload, direct forecasts and priors 6

6. Predictive uncertainty and distributions 6

7. Betting lines, pushes and fair odds 7

8. Supported player markets and deliberate gaps 7

9. FanDuel page behavior 8

10. Persistence, auditability and re-runs 8

11. Standalone use and canonical inputs 9

12. Validation scope and remaining research 9

0. What runs, how much history, and new seasons

This package implements the supplied **Weighted Historical Performance: NBA, NFL and NCAAF** research document as a player-game forecasting baseline. It is separate from the team-score models in `models/Game_Outcome`. The original research PDF is retained unchanged; `WHP_Implementation_Guide.pdf` explains the code in this folder. None of the source document's instructions trigger wagers, messages, or API requests.

Default	NBA	NFL	NCAAF
Source seasons loaded	Previous and current	Previous and current	Previous and current, selected schools
Maximum appearances in current weighted estimate	20	16	12
Exponential half-life, appearances	5	3	3
Minimum appearances before a point forecast	5	3	4
Minimum earlier walk-forward errors to quote probabilities	6	4	4
Maximum candidate games for error calibration	40	40	40
Prior strength k, when a disjoint older prior exists	5	5	8
Direct forecast blending weight beta	1	1	1
Maximum age of latest player observation	450 days	450 days	450 days

These are explicit starting configurations, not optimized or proven values. A point estimate alone does not authorize a probability: enough historical next-game forecast errors must also be available. With games sufficiently separated for results to be available, the smallest total histories are 11 NBA, 7 NFL and 8 NCAAF target observations. Additional missing values or availability lags can increase those requirements. The 40-game calibration cap means at most 40 **candidate appearances**, of which some can be ineligible for lack of earlier training data.

At a new season, the model does not discard the previous season. It takes the newest eligible appearances across the two loaded seasons, up to the configured window. Thus an NBA player with two games this season can use up to 18 appearances from last season; an NFL player with two can use up to 14; a college player with two can use up to 10. Dates, seasons and IDs must remain distinct. Offseason months are not extra appearances and do not automatically decay the default appearance-based weights. Calendar decay can be configured explicitly, in which case half-life is measured in days.

Older eligible observations **outside** the current window form a player-specific empirical prior only if at least three exist. They cannot overlap the weighted estimate. The prior uses the arithmetic mean of those older observations; all are still bounded by the two-season source scope. If that disjoint pool is absent, alpha is one and the engine leaves the weighted mean unshrunk. It never supplies a fictional rookie mean or silently pools quarterbacks with receivers. A rookie with insufficient observations/calibration remains unavailable. Larger k makes college shrinkage stronger when a prior actually exists; it cannot create information when there is no prior.

NBA/NFL identities can carry across a trade if the stable source player ID remains the same and the newest saved team matches the selected matchup. NCAAF preparation downloads only the two selected schools, so a transfer's previous school may not be covered. Add verified canonical history for that ID if cross-school carryover is needed; names alone do not merge IDs. Unmatched or ambiguous player names remain unavailable.

1. Package boundaries and execution flow

File	Responsibility
`weights.py`	Uniform, custom, linear, exponential and calendar weights; mean, variance and effective N
`opportunity.py`	Ratio-of-weighted-totals efficiency and explicit projected-workload input; college carry-share utility
`core.py`	Configurations, chronological eligibility, direct/shrunk/workload means, walk-forward dispersion and forecast object
`distributions.py`	Integer-binned normal, Poisson/negative-binomial and weighted empirical settlements
`markets.py`	Exact bookmaker key to source statistic and exposure mappings
`historical_data.py`	Saved NBA/NFL history, CFBD college box scores, availability proxies and canonical imports
`prediction_service.py`	Resolve players, create market targets, freeze forecasts, compare saved quotes and write audits
`market_comparison.py`	Raw implied probability, two-sided no-vig probability and refund-aware expected return
`backtesting.py`	Forward replay, MAE/RMSE/bias, Brier, log loss and reliability bins
`__main__.py`	Explicit preparation, standalone pricing and backtest commands
`errors.py`	Invalid/insufficient-data failures

The dashboard calls `sports_analytics/whp_service.py`, which exports `WHPMarketService`. Calculation and collection code remain in the player methodology folder. Reading markets never invokes the Odds API. WHP reads local player observations, generates a single forecast distribution per player/target/cutoff, and reuses that distribution across all alternate thresholds and both sides.

2. Source observations and preparation

NBA history reuses the existing NBA Stats game-log adapter and SportsDataverse historical box-score fallback. NBA season numbers are start years: 2025 means 2025-26. October starts a new NBA season in this application. NFL history reuses nflverse weekly player statistics plus its schedule for dates. Football season rolls in March, so January 2027 belongs to season 2026. Both regular-season and postseason datasets are eligible and are requested separately during preparation.

The **Prepare player history** button first reuses saved source datasets. Missing NBA/NFL datasets, or saved datasets with missing game dates, are downloaded through the existing adapters. Select **Refresh saved source data** to explicitly refresh existing observations; normal WHP renders only read saved data. Older NFL caches with missing dates can also recover dates by game ID from the already-saved nflverse `data/modeling/nfl/raw/games.csv`, without a request. Refresh failures can leave earlier cached observations available with a notice. Missing/postseason-with-no-games datasets are reported rather than filled with synthetic rows.

NCAAF uses a dedicated CFBD adapter without restoring the Historical Data player mode. The preparer resolves the selected full team names through `/teams/fbs`, then requests `/games` and `/games/players` for each school and each of the two seasons with `seasonType=both`. It retains only games explicitly marked completed, joins by game ID, and pivots per-athlete passing/rushing/receiving categories. Typical first preparation uses two school-directory requests plus eight school-season requests. CFBD access and quotas depend on the configured account. No Odds API credit is used for these requests. Data returned for the opposite school in the same box score is filtered to the requested school.

CFBD fields include passing YDS/TD/INT and C/ATT, rushing CAR/YDS/TD, receiving REC/YDS/TD. Compound `20/30` becomes 20 completions and 30 attempts. Explicit `0` remains an observed zero. An absent target, absent category, malformed number or DNP is not converted to zero. Standard college boxes do not provide the complete targets, all-scorer TDs or scoring-order inputs required for every proposed extension.

NBA archive player records without positive minutes are excluded by the existing adapter. NFL and college histories are conditional on recorded participants. There is no separate probability of playing, returning from injury, starting, or receiving a particular role. Box-score selection can underrepresent zero-opportunity appearances; use authoritative participation imports when necessary. Interpret the distribution as conditional on the represented appearance population, not an unconditional roster forecast.

Official interface references:

- NBA Stats: https://www.nba.com/stats/players/boxscores
- SportsDataverse releases: https://github.com/sportsdataverse/sportsdataverse-data/releases
- nflverse releases: https://github.com/nflverse/nflverse-data/releases
- nflverse player statistics: https://nflreadr.nflverse.com/reference/load_player_stats.html
- CFBD box-score schema: https://api.collegefootballdata.com/api/games
- CFBD school directory: https://api.collegefootballdata.com/api/teams
- Odds API market definitions: https://the-odds-api.com/sports-odds-data/betting-markets.html
- NCAAF event-odds interface: https://the-odds-api.com/sports/ncaaf-odds.html

3. Cutoffs, identity matching and data quality

For a saved snapshot, `as_of = min(snapshot.pulled_at, snapshot.commence_time)`. A snapshot taken during a game therefore uses a pregame baseline, not the ongoing box score. Each observation must have an event date and result-availability timestamp strictly earlier than this cutoff. Within every calibration replay, the same checks use the held-out game's start as the cutoff. Future results and target-game realized minutes or attempts do not enter its forecast.

Public box scores do not provide a complete archive of original publication times or later corrections. In the absence of an explicit `available_at`, the adapter uses a conservative proxy: eight hours after an explicit start, or the next UTC day for a midnight date-only observation. This is clearly recorded in forecast diagnostics. It reduces same-game leakage but is **not a certified point-in-time feed**. Later revised box scores may still differ from information originally available. Supply audited availability times in imports for stronger historical testing.

Player names are Unicode-normalized, punctuation-normalized and stripped of Jr/Sr/II/III/IV suffixes, then matched exactly. No fuzzy matching is used. The name must resolve to one stable player ID whose newest eligible team matches the matchup. NBA/NFL full team names map to codes, with relocation/abbreviation aliases. If two IDs share the name on eligible teams, the model refuses to choose. Full college names are mapped to canonical school names during source preparation.

Targets must be observed finite integers; signed football net yardage is allowed. Duplicate player/game IDs with identical full rows are removed. Conflicting duplicates or overlapping imports are rejected; the engine does not pick arbitrary revisions. Missing target rows are excluded from that target's history and effective sample; their values are not imputed. Combination targets require every component in the same game.

4. Recency weighting and weighted dispersion

The current appearance window is chronological, oldest to newest. For exponential weights, age is N-1 through zero and `raw_i = 2 ** (-age_i / half_life)`. Weights are normalized to sum to one. The newest observation therefore has the highest weight. Calendar weighting replaces appearance age with elapsed days. Uniform and linear alternatives implement the PDF's SMA and `2*i/[N*(N+1)]` formulas; custom normalized weights are available through the low-level weights API.

`weighted_mean = sum(w_i * X_i)` and `weighted_variance = sum(w_i * (X_i - weighted_mean)**2)`. The finite-sample corrected descriptive variance divides by `1 - sum(w_i**2)`. Effective sample size is `1 / sum(w_i**2)`. The engine records all of these. Corrected descriptive variance remains a descriptive assumption under dependence, not a substitute for held-out forecast calibration.

The PDF's five-game example produces a mean of 25.8 from [15,18,20,27,30] and custom weights [.05,.10,.15,.25,.45]. Effective N is 3.333333. With prior 20 and k=5, alpha is .4 and the adjusted mean is 22.32. The ten-game exponential example at half-life three produces mean 26.108708, descriptive SD 3.746163 and effective N 7.125124. Automated tests reproduce these values.

5. Workload, direct forecasts and priors

The opportunity module implements `rate = sum(w*stat)/sum(w*opportunity)`, followed by `rate_mean = rate * projected_opportunity`. It never averages individual per-minute rates, which would overemphasize tiny stints. NBA opportunities include minutes and three-point attempts; football uses pass attempts, carries and targets when provided. Completions/TDs/interceptions can use attempts as exposure. Receiving yards per target is algebraically the catch-rate times yards-per-reception decomposition when all components use identical samples and weights.

If no external workload projection is provided, the exposure projection is its weighted historical mean. Multiplying it by the weighted aggregate rate then simplifies to the direct weighted statistic mean. The code and diagnostics preserve that identity rather than claiming a new signal. The default beta is one, so the dashboard uses the direct WHP benchmark, followed by a disjoint prior when available. It does not invent injury/news/minutes projections or advertise fitted workload coefficients.

Researchers can supply a projected opportunity and choose beta below one through the engine/CLI. Then `blended = beta*direct + (1-beta)*rate_mean`. If a disjoint prior exists, `alpha = effective_N/(effective_N+k)` and `adjusted = alpha*blended + (1-alpha)*prior`. Without a prior, alpha is one. Missing or zero historical exposure prevents an explicit rate override rather than dividing by zero. Overrides must be known before the cutoff. Historical replay never feeds actual target-game exposure back as a projection; the default replay uses the weighted opportunity baseline. Override-based uncertainty therefore needs additional scenario validation.

NCAAF's optional carry-share utility implements `player_share = sum(w*player_carries)/sum(w*team_carries)` and `projected_player_carries = player_share*projected_team_carries`. It checks that player carries are between zero and team carries. Team volume, opponent-strength, news and expected rotation modules are not connected automatically; the utility is available for explicit researched inputs.

6. Predictive uncertainty and distributions

For each of up to 40 candidate historical appearances, the engine recreates the point forecast from only results available before that game's start. Once the minimum earlier history is available, it records the predicted mean and realized statistic. At least the configured number of such errors is required. Predictive MSE is the mean squared next-game forecast error; normal predictive variance is `max(0.25, MSE)`. The small floor is integer-bin resolution for an otherwise constant series. It is not a claim of measured uncertainty at that floor.

This predictive dispersion is **not** descriptive SD divided by square root of effective N. It targets next-game outcomes rather than estimation precision of the mean. The error record also reports MAE, RMSE and forecast-minus-actual bias. Existing bias is conservatively included in RMSE; no extra intercept correction is silently fitted.

Target	Default distribution	Implementation limit
NBA points and combinations containing points	Integer-binned normal	Rounded, left-censored at zero; approximate tails
NBA rebounds, assists, threes, steals, blocks, turnovers and rebound-assist combo	Poisson or negative-binomial count	Count approximation; no bounded-attempt binomial model
NFL/NCAAF net yards	Integer-binned normal, signed	Can represent negative net yards; no explicit yardage mixture
Football attempts, completions, interceptions, receptions and TD counts	Poisson or negative-binomial count	Counts can be overdispersed; not a joint attempts/success simulation

For count targets, variance is the maximum of predictive MSE, corrected weighted descriptive variance and the projected mean. Poisson is used when variance is effectively the mean. Otherwise the negative-binomial size is `r = mean**2/(variance-mean)`. The implementation sums log-space PMFs to avoid simple underflow at moderate large means. Nonnegative target means and finite parameters are enforced. Zero mean is a degenerate zero-count distribution.

Weighted empirical settlement is an explicit configurable alternative. It uses the observed sample and weights directly and cannot extrapolate beyond the observations. A shrunk/opportunity-adjusted point mean does not shift empirical support; this difference is intentional and visible in the configured family. The live defaults use normal/count distributions. Separate beta-binomial threes, zero-inflated yardage, per-reception maximum simulations and opponent-adjusted mixtures remain future extensions.

7. Betting lines, pushes and fair odds

For Over L, win means X>L, loss means X<L and an integer equality is a push. Under reverses win and loss. With integer-bin normal approximation, `P(X<=k) = NormalCDF(k+0.5)`. Thus a 24.5 line has no push, while a line at 25 can refund exactly 25. Negative net yards retain their signed support. Nonnegative normal targets collect the lower tail at zero instead of allocating negative outcomes.

The market probability is `win/(win+loss)` and excludes pushes. Fair American odds are `+100*(1-p)/p` for p<=.5, or `-100*p/(1-p)` above .5. Endpoints have no finite two-sided price, displayed as unavailable rather than a made-up finite quote. Hover text displays the unconditional win, loss and push probabilities too.

Market implied probabilities use `100/(A+100)` for positive A and `abs(A)/(abs(A)+100)` for negative odds. They include bookmaker margin. Edge in the UI is **WHP conditional probability minus raw market implied probability**, measured in percentage points, consistent with existing game markets. If matching Over/Under or Yes/No prices exist for the same player/key/line/snapshot, the audit also records proportional no-vig probability. It never pairs different thresholds or assumes that an absent opposite price exists.

Expected return per dollar is `win*payout - loss`, with push returning the stake. The audit stores this alongside raw/no-vig probabilities; expected value does not trigger any wager. Two-sided price pairing is diagnostic; live Edge continues to use the raw break-even price requested by the application.

8. Supported player markets and deliberate gaps

NBA maps the existing eleven alternate prop families: points, rebounds, assists, threes, blocks, steals, turnovers, points-assists, points-rebounds, rebounds-assists and points-rebounds-assists. Standard keys without `_alternate` are also recognized by the model. Combination observations are summed **within each game** and calibrated as their own target. This preserves historical covariance instead of adding independent marginal variances.

NFL/NCAAF map passing yards/attempts/completions/TDs/interceptions, rushing yards/attempts/TDs, and receiving yards/receptions/TDs. NFL `passing_interceptions` is canonicalized to `interceptions`. Position labels are not used to fabricate observations; a market needs its exact statistic and sufficient target history. All supplied alternate lines use the same forecast for monotone probabilities.

Anytime scorer and total scoring-TD props require an authoritative `touchdowns` column containing **all credited player scoring TDs** according to bookmaker settlement, including applicable returns/recoveries. Passing touchdowns do not mean that the quarterback personally scored. The standard adapter does not manufacture an all-scorer total by summing potentially overlapping defensive/fumble/return columns. Supply an audited canonical target to enable those probabilities. Without it, the actual market odds remain visible and WHP explains the missing target.

First and last scorer need scoring-sequence information and competing-player/team modeling, which a player-game average does not contain. They stay unpriced. Longest plays need per-play maxima and are not implemented. Unsupported definitions, incomplete components, missing history or ambiguous IDs always show unavailable with an explanation. There is no substitution of another player, another statistic, or another forecast method.

9. FanDuel page behavior

Open any NBA, NFL or NCAAF game, choose an existing snapshot or explicitly click Pull Live Markets, then open a player-prop category. WHP is the player forecast method; the separate PPP/EPA/ELO selector continues to govern Game Lines for NBA/NFL. College game-outcome methods remain unavailable and are labeled accordingly.

Player rows show Selection, Market odds, Market implied, WHP probability, WHP fair odds and Edge. Hover the model cells for mean, predictive SD, effective N, window size, calibration count, family, cutoff and source failures. Apply Criteria preserves the current player tab, evaluates all calculated selections in that category, highlights qualifying rows/sections in yellow and evaluated no-match sections in red, and shows counts. Defaults remain probability above 75% and Edge above five percentage points. Both are strict inequalities. Extra selections remain collapsed until Show more; Show less appears at the bottom after expansion.

All three leagues reuse saved CSV snapshots. NCAAF uses a separate `ncaaf_snapshots` SQLite table and `ncaaf_...csv` names; it shares the existing atomic export, duplicate-click guard, account scoping and API credit log. College pulls request the existing 17 football market keys at FanDuel only; unavailable markets are not fetched separately. Actual credit use comes from response headers, and coverage can vary by bookmaker, event and jurisdiction. No forecast, criterion action, preparation button or expansion requests odds.

10. Persistence, auditability and re-runs

NBA/NFL source observations remain in the existing `data/cache/sports.sqlite3`, using the same query keys as Historical Data. College player sources live under `data/modeling/player_stats/whp/ncaaf`. Canonical imports live under `.../whp/imports/<sport>.csv`. These files are local and ignored by Git. Input files contain player/game IDs and observations, not credentials.

Each forecast is frozen under `.../whp/forecasts/<sport>/<hash>.json`. The identity includes sport, player, target, pregame cutoff, configuration, eligible input fingerprint and calculation/source-mapping code hashes. Changes to source observations, code, cutoff or parameters create a distinct cache identity. All alternate thresholds reuse that distribution. Forecast JSON stores mean, predictive variance, family, observed sample, weights, source availability assumptions and every calibration prediction.

Quote comparison audits are saved under `.../whp/evaluations/<hash>.json`. They record the saved snapshot identity, cutoff, model implementation hash and per-line win/loss/push, conditional probability, fair price, raw market price, Edge, optional no-vig probability, expected return and explanations. Original market CSVs remain unchanged. Atomic temporary-file replacement prevents partial JSON writes. Reopening a snapshot with identical code/inputs restores the frozen estimates without fitting again or downloading history.

Changing a source dataset can change a later replay even for an old snapshot if the source corrected history; the existing saved forecast/audit remains archived under its original input hash. True as-originally-known historical replay additionally requires versioned provider inputs and original availability timestamps.

11. Standalone use and canonical inputs

Run from the repository root. No new runtime dependency is required beyond `requirements/requirements.txt`; the count and normal distributions use NumPy and the Python standard library.

```
python -m models.Player_Stats.Weighted_Historical_Performance --sport NBA --prepare --start
    2026-10-24T23:00:00Z
python -m models.Player_Stats.Weighted_Historical_Performance --sport NFL --prepare --start
    2026-10-24T23:00:00Z
python -m models.Player_Stats.Weighted_Historical_Performance --sport NCAAF --prepare --start
    2026-10-24T23:00:00Z --teams "Alabama Crimson Tide" "Auburn Tigers"
```

For an individual canonical target CSV, required columns are `game_id,event_date,available_at,value`. Optional `opportunity` enables a supplied workload projection. Use one player/target per CLI input. Dates include timezones and values are integer observed game results. Model service imports need the wider schema described in `INPUT_SCHEMA.md`.

```
python -m models.Player_Stats.Weighted_Historical_Performance --sport NBA --history player_points.csv
    --as-of 2026-10-24T12:00:00Z --line 24.5
python -m models.Player_Stats.Weighted_Historical_Performance --sport NFL --history passing_yards.csv
    --as-of 2026-10-24T12:00:00Z --line 249.5 --signed
python -m models.Player_Stats.Weighted_Historical_Performance --sport NBA --history player_points.csv
    --as-of 2026-10-24T12:00:00Z --line 24.5 --beta 0 --opportunity 34
python -m models.Player_Stats.Weighted_Historical_Performance --sport NBA --history player_points.csv
    --as-of 2026-10-24T12:00:00Z --backtest --line 24.5 --evaluation-start 2026-01-01T00:00:00Z
```

The forecast command prints audit-ready JSON. The research API exposes `WHPConfig`, `WeightedHistoricalPerformance`, `WHPForecast`, `recency_weights`, `rate_projection`, `carry_share`, `walk_forward_backtest` and `WHPMarketService`. `predict()` reproduces the source PDF's weighted estimator independently from probability fitting; `forecast()` enforces chronological data and minimum error calibration.

12. Validation scope and remaining research

Unit tests reproduce the PDF examples and validate recency weighting, workload algebra, count/normal/empirical settlements, signed yardage, integer pushes, prior-season carry, minimum samples, duplicates, future-result exclusion, canonical identity matching, combination covariance, CFBD source parsing, all-three-league persistence, paid-pull deduplication, credit sport labels and actual Dash callbacks. Browser verification covers prop columns, criteria and expansion with saved/local inputs. None of those checks spends Odds API credits.

`walk_forward_backtest` repeatedly fits the entire uncertainty pipeline from earlier observations, then evaluates the next unseen result. It reports MAE, RMSE, bias, settled-game Brier/log loss and ten reliability bins. Integer pushes are excluded from conditional binary probability scores. Set `evaluation_start` to reserve a final test interval. The CLI also truncates all data at `--as-of` before replay. The utility does not optimize hyperparameters; if searching half-lives/windows/beta/families, use earlier folds and preserve the final interval.

Walk-forward calibration is implemented, but the research starting configurations have not passed a league-wide untouched validation study. Injury/rotation forecasts, opponent strength, weather, team-play projections, multi-player dependence, betting sequence, max-play simulation and real point-in-time publication archives are not automatic features. Treat reported probabilities as transparent research estimates; missing inputs remain explicit rather than being disguised as model output.

Build provenance

The following SHA-256 hashes identify the reviewed calculation/service files. They are documentation provenance, not a runtime cache identity.

models/Player_Stats/Weighted_Historical_Performance/README.md
`7674f7c4f7897c5db2884d9d3fcdd6395de460c3ba3aea0138b13b4764812d71`

models/Player_Stats/Weighted_Historical_Performance/__init__.py
`3d7bcde36cb78645c37f5e67a964d95ffe9a70b5065ca7b9a067557e587ce98b`

models/Player_Stats/Weighted_Historical_Performance/__main__.py
`4f51a7aa7ae20a5b54a561aecc0f0b8891e5157f9b6b246d6fc96165e92cf179`

models/Player_Stats/Weighted_Historical_Performance/backtesting.py
`ded5f490f42ee32e4f6fac3a07e0895137bd528f7466ed4e7db5ac6c32ed0825`

models/Player_Stats/Weighted_Historical_Performance/core.py
`3cd8ac53d9cba9489c38bf2315a7165ff411cb23c772ede89a0c9d6f38abf78f`

models/Player_Stats/Weighted_Historical_Performance/distributions.py
`c4b921a155ef7b62b46f6a4ff3f376ad2b4ef6cf09e3ad1b6889eb14adb3dcbe`

models/Player_Stats/Weighted_Historical_Performance/errors.py
`afb5ade277ba24e5fbf77380ebc128570e7de5f737647f31d9c7b01b3845bb6e`

models/Player_Stats/Weighted_Historical_Performance/historical_data.py
`32a1e70d69f540fad6ba6d382cf662fb30dd7448aa649300d0985c8af5f67350`

models/Player_Stats/Weighted_Historical_Performance/market_comparison.py
`305f3f60a80b2c0e073a84d7b341944796562e2d9589df612dcd6be216c919d8`

models/Player_Stats/Weighted_Historical_Performance/markets.py
`65df9689ed1dd05b84eacb490059e7055ca737aedd7d9d435243c03775ac0e68`

models/Player_Stats/Weighted_Historical_Performance/opportunity.py
`85d912906e1ff65c30075962a244db70902b37dff6bffab8df19c0059c5ff4bd`

models/Player_Stats/Weighted_Historical_Performance/prediction_service.py
`65017888cbf40991e999c169c0dfe4a4aaf3fe0c9657d5400315626253b1d035`

models/Player_Stats/Weighted_Historical_Performance/weights.py
`3d06c0c7cda1dd922aa1839528fd730b28642d537aaada1f9a8c0ab33dcc4ecc`